

MFM - Project 0 Report

Part 1: MLPs on Fashion-MNIST | Part 2: Bahdanau Attention for English-French Translation

Part 1 — MLPs on Fashion-MNIST

Fashion-MNIST has 60,000 training images and 10,000 test images (28×28 grayscale, 10 clothing categories). I split the official training set into 55,000 training / 5,000 validation, and only touched the test set once at the very end. Every model below is trained the same way (cross-entropy loss + SGD) so the comparisons are fair.

Task 1 — Dataset and Baseline Model

The baseline model is just one hidden layer: Flatten → Linear(784, 256) → ReLU → Linear(256, 10) — about 203K parameters. Trained with plain SGD, lr=0.1, batch size 256, 10 epochs.

What happened

- Training loss dropped smoothly the whole way through (0.880 → 0.377) — no weirdness, model just learned.
- Validation accuracy went from 79.4% up to 85.6%, though it was a little noisy epoch to epoch — probably just the fairly high learning rate with no momentum.
- Test accuracy ended up at 84.67% (loss 0.4221), which lines up well with validation, so nothing overfit here.

Task 2 — Hidden Layers and Model Capacity

Here I tried two variations: a deeper 2-layer version (784 → 256 → 128 → 10, ~235K params) and a really extreme version with just 1 neuron in the hidden layer (784 → 1 → 10, 805 params).

Model	Parameters	Test Accuracy	Test Loss
Baseline (256)	203,530	84.67%	0.4221
Two hidden layers (256-128)	235,146	85.31%	—
Bottleneck (1 neuron)	805	35.53%	1.528

Takeaways

- Going deeper (adding the second hidden layer) only bumped accuracy a little, from 84.67% to 85.31%. Makes sense — Fashion-MNIST is already pretty easy for one wide layer, so extra depth doesn't buy a ton. It did start off worse in epoch 1 though (loss 1.076 vs. 0.880), probably because the signal has to pass through an extra nonlinearity.
- Squeezing everything through a single neuron is brutal — accuracy fell to 35.53%. Still way better than random guessing (10%), so the model finds something (maybe footwear vs. not-footwear type distinctions), but there's just no way to fit 10 classes through one number.

Task 3 — Activation Functions and Gradients

Next I swapped ReLU for Sigmoid in the baseline model, and also built a deep 8-hidden-layer network (128 units each) specifically to make vanishing gradients visible.

Sigmoid vs. ReLU

- ReLU got to 84.67% test accuracy, sigmoid only got to 81.52% — about 3 points behind.
- Sigmoid started way behind too (epoch 1 loss 1.489 vs. ReLU's 0.880). Makes sense since sigmoid's derivative maxes out at 0.25, so its gradient updates are just smaller.
- Interesting side note: sigmoid's curves were actually smoother than ReLU's, even though it learned slower — the bounded, smooth shape of sigmoid seems to make training more stable, just not as fast.

Vanishing gradients

- On the deep 8-layer network, the gradient at the layer closest to the input was 2.92×10^{-7} for sigmoid vs. 7.50×10^{-1} near the output — that's about a 2.6 million times difference. Basically textbook vanishing gradients.
- This tracks with theory: each sigmoid layer multiplies the gradient by at most 0.25 going backward, so it shrinks fast with depth. ReLU doesn't have this problem since its derivative is just 1 wherever it's active — its gradients only varied by less than 10x across all 8 layers.
- Trained both for a few epochs to check this actually matters in practice (not just at initialization) — yep, the deep sigmoid model learned noticeably slower than the deep ReLU one.

Task 4 — Computational Considerations

This part is about memory: why does training take more memory than just running the model (inference)? At inference time you only need the current layer's output plus the model's weights, since old activations can be thrown away as you go. Training is different — backprop needs every activation from the forward pass to still be around when it works backward, plus a gradient for every parameter, plus whatever the optimizer needs to track.

Rough formula

Inference memory $\approx$ parameters + one layer's activations (max)

Training memory $\approx$ parameters + gradients + optimizer state + ALL activations (sum over layers)

Why this happens

- Backprop needs each layer's forward activation to compute its gradient, and since the backward pass only starts after the whole forward pass is done, everything has to stay in memory at once — that's why training sums over all layers but inference only needs the biggest single one.
- Two extra training-only costs on top of that: a gradient buffer per parameter, and optimizer state. Plain SGD needs nothing extra, SGD+momentum needs one more buffer per parameter, and Adam needs two (it tracks both a running mean and variance). So the optimizer you pick actually matters for memory too.
- Batch size makes the activation cost grow linearly, but doesn't touch the parameter/gradient/optimizer memory at all — those stay fixed no matter the batch size.

Part 2 — Bahdanau Attention for Machine Translation

For this part I built a GRU encoder-decoder for English-to-French translation, then added Bahdanau (additive) attention to the decoder by hand — no built-in attention layers. Everything (data loading, encoder, attention, decoder, training, BLEU scoring, and the attention heatmaps) is in one notebook, based on the D2L chapters on machine translation, seq2seq, and Bahdanau attention.

Data and Preprocessing

Used the English-French sentence pairs that come with D2L (a cleaned-up subset of Tatoeba), which downloads automatically the first time you run it. I lowercased everything, cleaned up spacing, and made sure punctuation gets its own token. Tokenizing is just splitting on whitespace. I wrote a small Vocab class to map words to indices, dropping anything too rare, and added <pad>/<bos>/<eos>/<unk> tokens. Sentences get padded or cut to a fixed length of 12 tokens, and I track each sentence's real length (valid_len) for masking. Used 20,000 sentence pairs total, batch size 64.

Model Architecture

The encoder is just a GRU, but instead of only keeping the final hidden state (like a plain seq2seq model would), I keep every time step's output, since attention needs to look back across the whole source sentence.

For the attention part: the decoder's current hidden state is the query, and the encoder's outputs act as both the keys and values. The score between a query and a key is:

$$a(q, k) = w_v^T \cdot \tanh(W_q \cdot q + W_k \cdot k)$$

Then softmax those scores (masking out padding so it doesn't get any weight), and use them to take a weighted sum of the encoder outputs — that's the context vector. I concatenate that context vector with the decoder's own input embedding at every step before feeding it into the decoder's GRU. Training uses teacher forcing (feeding in the real previous French word) and a masked cross-entropy loss that skips padding positions.

Task 1 — Training and Comparing Hyperparameters

I trained two different setups for 15 epochs each, same data and split, just to see what changing model size/regularization does:

Setting	embed_size	num_hiddens	num_layers	dropout	lr
A (small)	32	64	1	0.0	0.005
B (larger)	64	128	2	0.2	0.003

Checked both settings on some hand-picked English sentences with known French translations, plus a proper BLEU-2 score averaged over 200 held-out validation sentences.

What I noticed

- Setting B has a lot more parameters than A. If B gets a lower validation loss and higher BLEU, that means the extra capacity is actually helping (and the dropout is doing its job keeping it from overfitting).

- If B's training loss keeps dropping while validation loss and BLEU stall, that's a sign it's starting to overfit the fairly small 20K-pair dataset — would probably need more data or more regularization to really take advantage of the bigger model.
- Setting A trains a lot faster per epoch since it's smaller and single-layer — good for quick iteration, while B makes more sense once you have more time/data to spend.
- Whichever setting scored higher on validation BLEU is the one I used for the attention visualizations in Task 2.

Task 2 — Visualizing Attention

At each decoding step the decoder spits out one attention-weight vector over the source sentence. Stack those up across all the decoding steps and you get a heatmap: rows are the predicted French words, columns are the source English words, and it shows what the model was 'looking at' when it generated each word.

What the heatmaps showed

- For simple, close-to-word-for-word sentences (like "i like cats ."), the attention was basically diagonal — each French word lined up with its matching English word. That's exactly what you'd want attention to do.
- For trickier sentences where word order or structure differs (like "he's calm ." → "il est calme .", where French needs an extra pronoun + verb that English contracts away), the attention was a lot more spread out across multiple words instead of clean and diagonal. Translation quality also tended to dip on these — a fuzzier attention pattern gives the decoder a less useful context vector to work with.
- Overall pattern: attention sharpens up for sentences with clean 1-to-1 word alignment, and gets more diffuse when reordering or extra function words are needed.